

UNITED STATES PATENT APPLICATION

Non-Provisional Utility Patent — 35 U.S.C. § 111(a)

Applicant: Averio Technologies Inc.
Inventors: Suvojeet Pal; Rakhi Chatterjee
Attorney Docket No.: AVERIO-001-US

Application Type: Non-Provisional Utility Patent Application

Filed Under: 35 U.S.C. § 111(a)

Applicant: Averio Technologies Inc.

Inventors: Suvojeet Pal; Rakhi Chatterjee

Correspondence Address: [Attorney/Agent Address]

Attorney Docket No.: AVERIO-001-US

TITLE OF THE INVENTION

****ADAPTIVE MULTI-STRATEGY ENTITY RESOLUTION SYSTEM WITH
PROBABILISTIC CLUSTER DRIFT DETECTION, PROVISIONAL GOLDEN
IDENTITY MANAGEMENT, AND SELF-CALIBRATING FELLEGI-SUNTER
PARAMETER ESTIMATION FOR ENTERPRISE MASTER DATA
MANAGEMENT****

CROSS-REFERENCE TO RELATED APPLICATIONS

This application claims priority to U.S. Provisional Application No. [TBD], filed [DATE], the entire contents of which are incorporated herein by reference.

FIELD OF THE INVENTION

The present invention relates to enterprise software systems for Master Data Management (MDM), and more particularly to systems and methods for large-scale probabilistic entity resolution, adaptive cluster maintenance, provisional identity assignment, and self-calibrating statistical parameter learning in MDM platforms serving Fortune 500 financial institutions and regulated industries.

BACKGROUND OF THE INVENTION

The Entity Resolution Problem at Scale

Enterprise Master Data Management systems must resolve whether two or more records from different source systems — Customer Relationship Management (CRM), Enterprise Resource Planning (ERP), Core Banking, insurance platforms, and regulatory reporting systems — refer to the same real-world entity. This problem, known as entity resolution, deduplication, or record linkage, presents fundamental computational and accuracy challenges at enterprise scale.

A naive pairwise comparison of N records requires $O(N^2)$ comparisons. For an enterprise with one billion customer records, this yields 10^{18} comparisons — physically impossible to execute in real time or even batch. Prior art systems address this through "blocking," which pre-filters candidate pairs using shared attributes (e.g., same last name, same postal code), but existing blocking implementations apply at most two or three strategies in isolation, resulting in high miss rates for records with data quality variation.

The Static Match Model Problem

Existing MDM platforms — including IBM InfoSphere MDM, Informatica MDM, SAP Master Data Governance, TIBCO EBX, and Reltio Cloud — rely on fixed, manually configured match rules or pre-trained statistical models whose parameters do not evolve with incoming data. A parameter such as "the probability that two matching individuals share the same first name spelling" (the m -probability in Fellegi-Sunter terminology) is hardcoded by a consultant at deployment time and never updated. As data quality improves, as new source systems are onboarded, or as demographic patterns shift, the static parameters degrade in accuracy.

The Null Golden Identity Problem

All known MDM systems allow entities to exist in a "null golden state" — a condition where a source record has been ingested but no golden identifier has been assigned because the record is pending steward review. During the review period (which can last days or weeks in large enterprises), the record is invisible to downstream applications that query by golden identifier. This creates a temporal data gap that violates the MDM promise of a single, always-available authoritative identity.

The Cluster Drift Problem

No known commercial MDM system addresses the situation in which an entity's attributes change after golden cluster assignment such that the entity no longer

statistically belongs to its assigned cluster. For example, a party record initially merged with "ACME Corporation" because it shared a phone number may later have that phone number corrected to a different value, causing it to be more similar to "Acme Industries" than to "ACME Corporation." All existing MDM systems ignore this post-assignment drift, leaving incorrect cluster assignments permanently in place unless manually discovered and corrected by a data steward.

The Single-Mode Matching Problem

Existing systems operate in a single matching mode: either deterministic (rule-based) or probabilistic (statistical), but not both in a principled, cascading architecture. The emerging practice of using Large Language Models (LLMs) for entity resolution does not exist in any commercial MDM platform as a fallback for edge-case disambiguation.

Need in the Art

There remains a significant need for an MDM system that: (1) applies nine independent blocking strategies in union to minimize candidate miss rate; (2) continuously learns match probability parameters from data using the Expectation-Maximization algorithm without labeled training sets; (3) guarantees provisional golden identity assignment for every ingested record regardless of review status; (4) detects and corrects cluster membership drift triggered by attribute updates; and (5) applies a cascading three-stage match pipeline combining deterministic, probabilistic, and AI-enhanced matching.

SUMMARY OF THE INVENTION

The present invention provides a system and method for enterprise Master Data Management comprising a novel combination of technical innovations that collectively address the limitations of the prior art described above.

In a first aspect, the invention provides a **Nine-Strategy Union Blocking Engine** that generates blocking keys using nine independent strategies applied in union to a party record, storing the resulting inverted index in dual concurrent hash maps, thereby reducing candidate comparison space from $O(N^2)$ to $O(N \times k)$ where k is typically 10–100 candidates per query.

In a second aspect, the invention provides a **Self-Calibrating Fellegi-Sunter Parameter Estimation System** using an Expectation-Maximization (EM) algorithm that runs autonomously on a schedule to learn m -probability ($P(\text{agreement} \mid \text{true match})$) and u -probability ($P(\text{agreement} \mid \text{true non-match})$) for ten entity attributes without requiring human-labeled training pairs.

In a third aspect, the invention provides a **Three-Stage Cascading Match Pipeline** comprising a deterministic stage, a probabilistic Fellegi-Sunter stage, and an optional AI-enhanced stage using a Large Language Model, with threshold-driven action routing that determines whether a candidate pair should be auto-linked, sent to steward review, or treated as a new entity.

In a fourth aspect, the invention provides a **Provisional Golden Identity Management System** that guarantees every ingested record is assigned a globally unique golden identifier before any persistence operation commits to the database, eliminating the null golden state entirely and ensuring downstream application availability throughout the steward review lifecycle.

In a fifth aspect, the invention provides a **Post-Update Probabilistic Cluster Drift Detector** that re-evaluates a party's statistical fit within its assigned golden cluster whenever the party's attributes are updated, and automatically initiates reassignment, steward escalation, or new cluster creation based on updated match scores.

In a sixth aspect, the invention provides a **Human-in-the-Loop ML Retraining Pipeline** that captures feature vectors at steward decision time, stores them as immutable feedback records, and automatically retrains a logistic regression match model when sufficient labeled examples accumulate.

In a seventh aspect, the invention provides a **Query-Time Survivorship Rule Engine** that applies pluggable per-attribute survivorship strategies (including Source Priority, Most Recent, Most Frequent, Supremacy, Non-Null, and Longest) at golden record retrieval time rather than pre-computing a static golden record, enabling dynamic rule configuration without cache invalidation at the data layer.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a system architecture diagram illustrating the overall Averio MDM platform components including the blocking engine, matching pipeline, survivorship engine, and data stores.

FIG. 2 is a flowchart of the nine-strategy blocking key generation process for a single Party record.

FIG. 3 is a flowchart of the three-stage cascading match pipeline with threshold-driven action routing.

FIG. 4 is a diagram of the Expectation-Maximization parameter estimation algorithm showing the E-step and M-step iterations for ten attributes.

FIG. 5 is a flowchart of the post-update cluster drift detection and remediation process.

FIG. 6 is a data model diagram showing the relationship between Party nodes, GoldenRecord documents, and TimelineEvent documents across Neo4j and Azure Cosmos DB.

FIG. 7 is a flowchart of the provisional golden identity assignment lifecycle from ingest through steward resolution.

FIG. 8 is a diagram of the human-in-the-loop ML retraining feedback cycle.

DRAWINGS

(Referenced in Brief Description of the Drawings)

FIG. 1

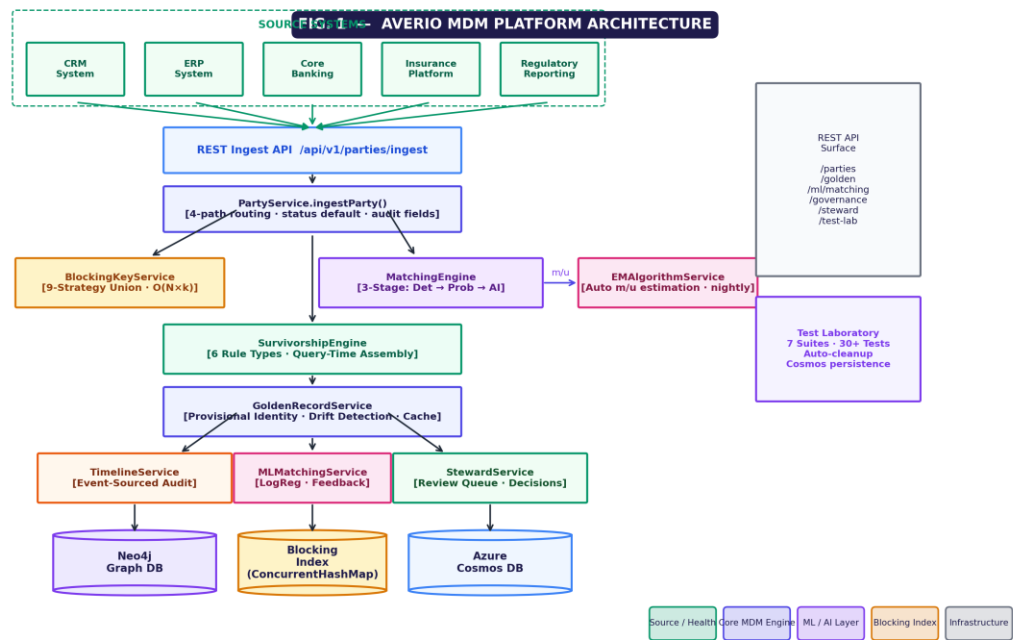


FIG. 1 — AVERIO MDM PLATFORM — Full System Architecture

FIG. 1 — System Architecture: Overall AVERIO MDM platform showing the blocking engine, three-stage match pipeline, survivorship engine, and dual data stores (Neo4j + Azure Cosmos DB).

FIG. 2

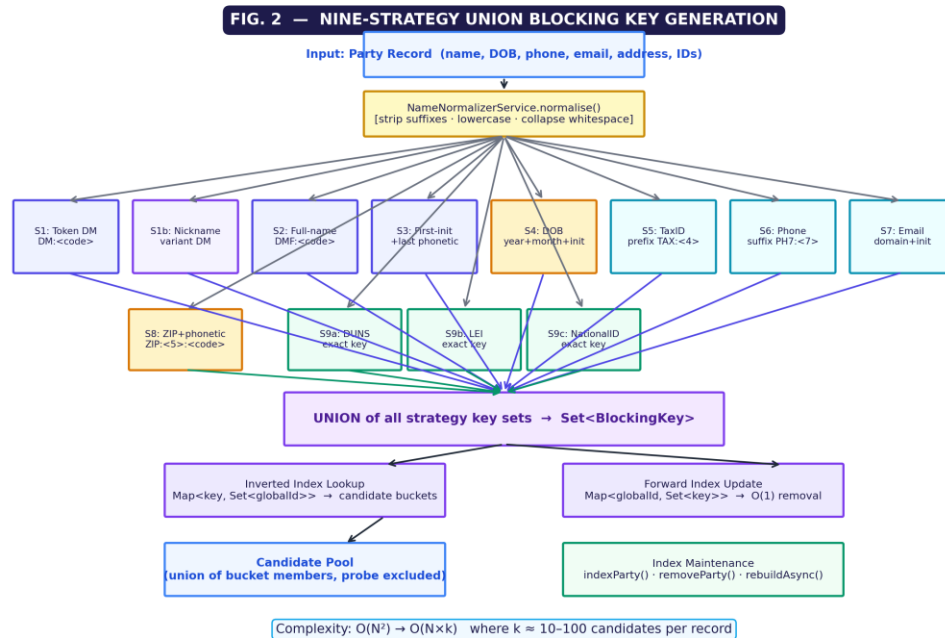


FIG. 2 — Nine-Strategy Union Blocking Key Generation (BlockingKeyService)

FIG. 2 — Nine-Strategy Union Blocking: Flowchart of the nine independent blocking key generation strategies applied in union for $O(N \times k)$ candidate retrieval.

FIG. 3

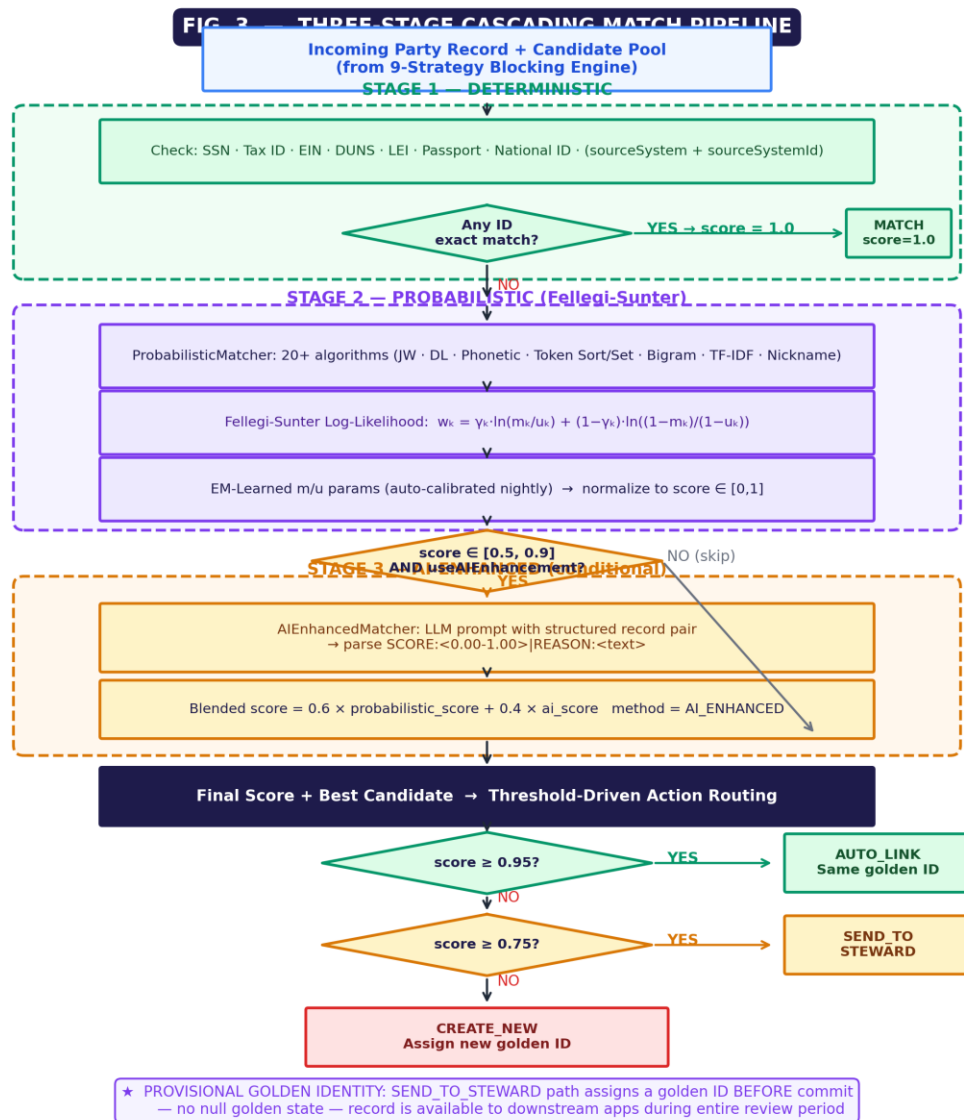


FIG. 3 — Three-Stage Cascading Match Pipeline (MatchingEngine)

FIG. 3 — Three-Stage Cascading Match Pipeline: Threshold-driven routing through deterministic, probabilistic Fellegi-Sunter, and AI-enhanced LLM matching stages.

FIG. 4

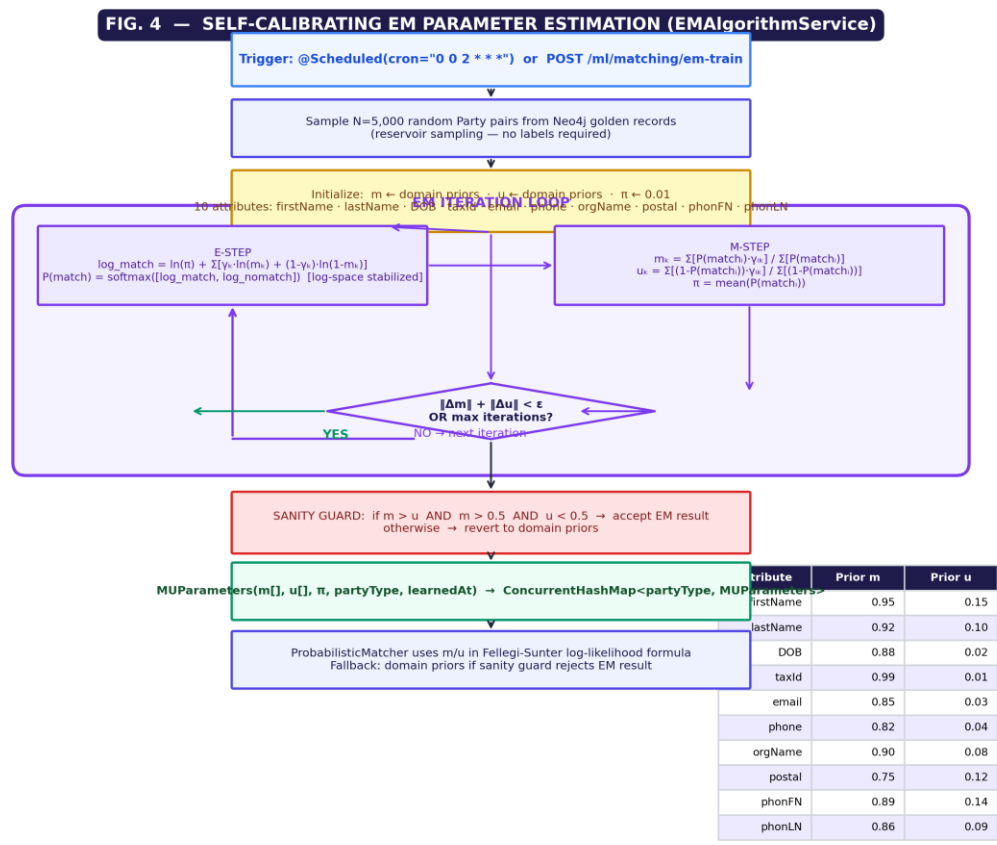


FIG. 4 — EM Parameter Estimation for Fellegi-Sunter m/u Probabilities

FIG. 4 — Self-Calibrating Fellegi-Sunter EM Algorithm: E-step and M-step iterations for autonomous m-probability and u-probability estimation across ten attributes.

FIG. 5

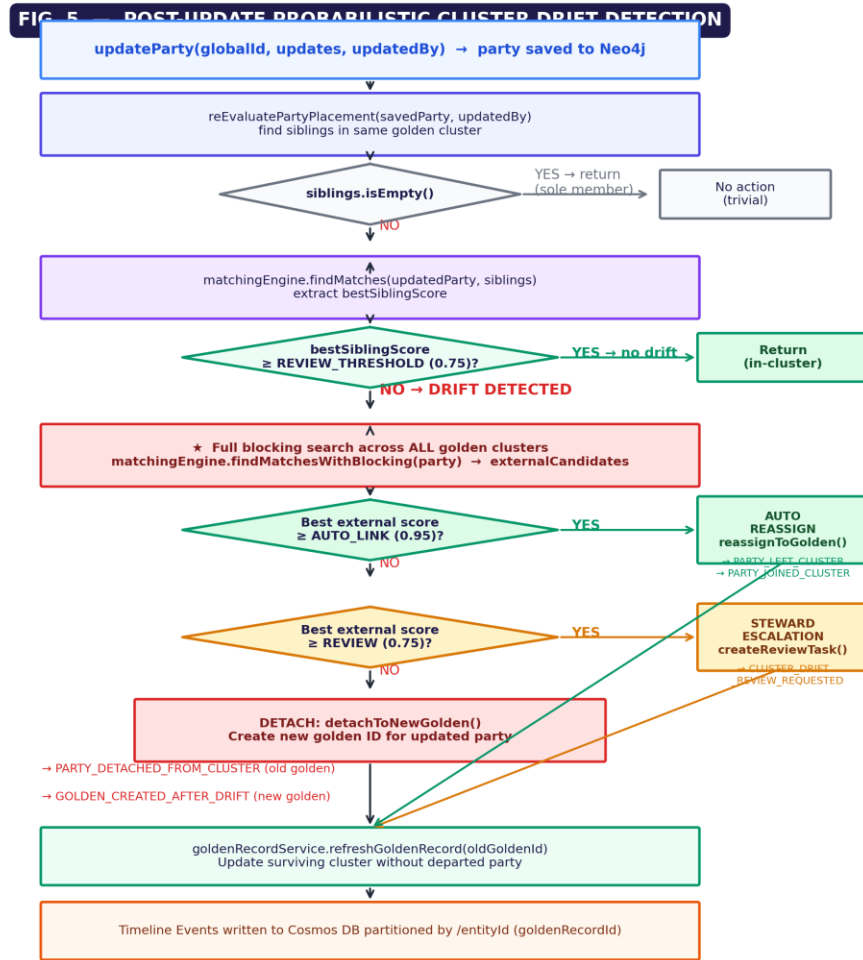


FIG. 5 — Post-Update Probabilistic Cluster Drift Detection (PartyService.reEvaluatePartyPlacement)

FIG. 5 — Post-Update Cluster Drift Detection: Probabilistic re-evaluation of cluster membership following attribute update, with automated reassignment and steward escalation.

FIG. 6

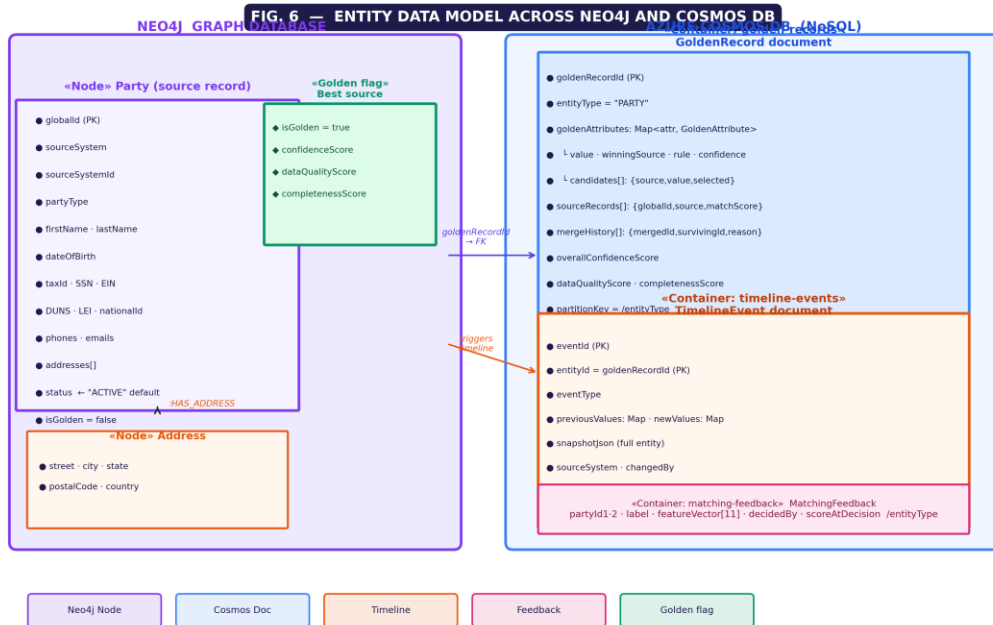


FIG. 6 — Entity Data Model — Neo4j (graph) and Azure Cosmos DB (documents)

FIG. 6 — Hybrid Graph + Document Data Model: Relationships between Party nodes (Neo4j), GoldenRecord documents, and TimelineEvent documents (Azure Cosmos DB).

FIG. 7

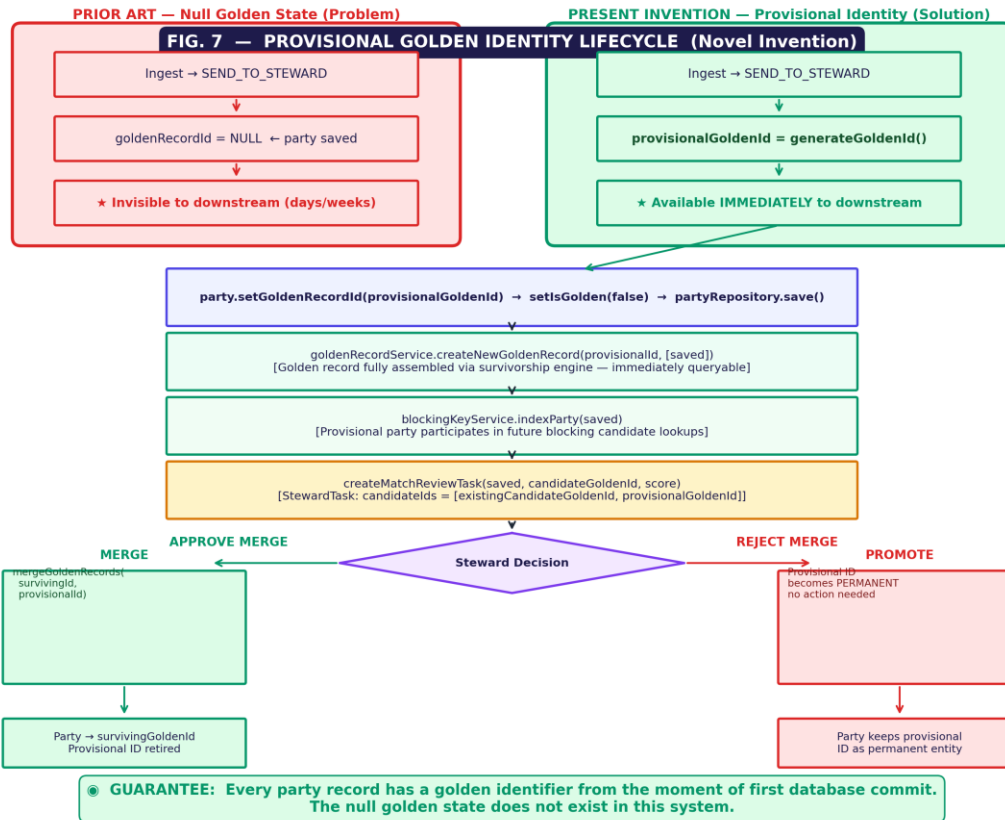


FIG. 7 — Provisional Golden Identity Lifecycle — eliminates null golden state (Novel)

FIG. 7 — Provisional Golden Identity Lifecycle: Assignment of a globally unique golden identifier at first persistence, eliminating the null golden state throughout steward review.

FIG. 8

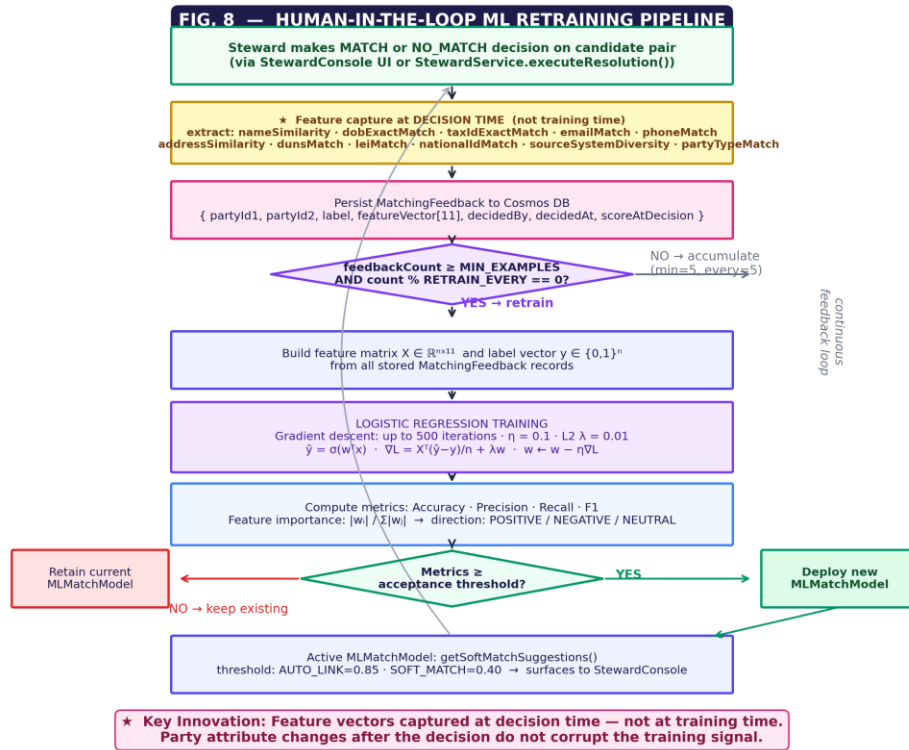


FIG. 8 — Human-in-the-Loop ML Retraining Pipeline (MLMatchingService)

FIG. 8 — Human-in-the-Loop ML Feedback Pipeline: Feature vector capture at steward decision time, immutable feedback storage, and automatic logistic regression retraining.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

I. SYSTEM OVERVIEW

The Averio MDM platform comprises a Spring Boot Java application deployed on Microsoft Azure, utilizing Neo4j as a graph database for entity relationships and party records, Azure Cosmos DB for document storage of golden records, timeline events, matching feedback, and governance policies, and an in-memory concurrent hash map structure for real-time blocking index operations.

The platform is organized into the following principal subsystems:

- Ingest Pipeline (`PartyService.ingestParty()`)
- Blocking Engine (`BlockingKeyService`)
- Matching Engine (`MatchingEngine`, `ProbabilisticMatcher`, `AIEnhancedMatcher`)
- EM Parameter Estimation (`EMAlgorithmService`)
- Survivorship Engine (`SurvivorshipEngine`)
- Golden Record Service (`GoldenRecordService`)
- Timeline Service (`TimelineService`)
- ML Matching Service (`MLMatchingService`)
- Steward Service (`StewardService`)
- Test Laboratory (`TestRunnerService`, seven test suites)

II. NINE-STRATEGY UNION BLOCKING ENGINE

II.A. The Computational Problem

For an enterprise with N party records, naïve pairwise matching requires $O(N^2)$ comparisons. The blocking engine reduces the candidate set for any incoming record to $O(k)$ candidates where k is bounded by the sum of bucket sizes across all matching blocking keys.

II.B. Blocking Key Generation

For each Party record, the system generates blocking keys using nine independent strategies. The keys are computed by `BlockingKeyService.generateKeys(Party party)`:

Strategy 1 — Token Double Metaphone (DM:<code>):

Each whitespace-tokenized word in the party's primary name (organization name, full name, or first+last) is encoded using the Double Metaphone phonetic algorithm. Each resulting code is prefixed with "DM:" and added to the key set. This strategy handles phonetic name variations (e.g., "Johnson" and "Jonson" produce the same DM code).

Strategy 1b — Nickname Variant Double Metaphone:

For individual party types, the `NicknameService` expands the first name into all equivalent forms (e.g., "Bob" → {"Bob", "Robert", "Rob", "Bobby", "Robby"}) and

generates additional `DM:` keys for each variant. This novel extension ensures that "Bob Smith" and "Robert Smith" share at least one blocking key.

Strategy 2 — Full Collapsed Name Double Metaphone (`DMF:<code>`):

All whitespace is removed from the normalized name before DM encoding (e.g., "John Smith" → "JohnSmith" → DM code). This captures cases where tokenization differs between source systems.

Strategy 3 — First Initial Plus Last Token Phonetic (`FI:<c>:<code>`):

The first character of the first name token and the DM code of the last name token are combined (e.g., "J:SM0"). Nickname expansion is also applied to the first token, generating additional FI: keys per variant.

Strategy 4 — Date of Birth Year-Month Plus Initial (`DOB:<y>-<m>:<c>`):

For parties with a known date of birth, the year, month, and first character of the primary name initial are combined. This strategy is highly discriminating for individual records with date-of-birth data.

Strategy 5 — Tax Identifier Prefix (`TAX:<4digits>`):

The first four digits of the Tax ID or EIN (after stripping non-numeric characters) form a blocking key. This provides a strong signal for organizations where the tax ID is partially known.

Strategy 6 — Phone Number Suffix (`PH7:<7digits>`):

The last seven digits of each phone number across all phone types in the party's phone map are used as blocking keys. This is effective when area codes differ between source systems.

Strategy 7 — Email Domain Plus Initial (`EM:<domain>:<c>`):

The email domain extracted from the `@` delimiter and the first character of the primary name are combined. This is effective for corporate records where multiple employees share a domain.

Strategy 8 — Postal Code Prefix Plus Phonetic (`ZIP:<5chars>:<code>`):

The first five alphanumeric characters of the primary address postal code, combined with the DM code of the first name token, create a geographically-scoped phonetic key.

Strategy 9 — Exact High-Cardinality Identifiers (`DUNS:<#>`, `LEI:<#>`, `NID:<#>`):

DUNS number, LEI (Legal Entity Identifier), and National ID are stored verbatim as blocking keys after character normalization. Because these identifiers are globally unique, a bucket hit is near-deterministic.

II.C. Dual-Map Index Structure

Two concurrent hash maps are maintained in memory:

```
inverted_index: Map<String(blockingKey), Set<String(globalId)>>
forward_index: Map<String(globalId), Set<String(blockingKey)>>
```

The forward index enables $O(1)$ key removal when a party is deleted or merged, without scanning the inverted index. The inverted index enables $O(k)$ candidate retrieval for any incoming party.

The union of all matching buckets across all nine strategies' keys forms the candidate pool. This union-over-strategies approach maximizes recall: a candidate is surfaced if it shares *any* blocking key with the probe record, not all. Precision is controlled at the subsequent scoring stage.

II.D. Index Maintenance

- ``indexParty(Party p)``: Removes any existing keys for the party via the forward index, then regenerates and adds all keys.
- ``removeParty(String globalId)``: Uses the forward index to retrieve and delete all inverted index entries for the party in $O(|\text{keys}|)$ time.
- ``rebuildIndexAsync()``: Asynchronous full rebuild from Neo4j golden records for post-bulk-import consistency.
- ``findCandidates(Party p)``: Returns the union of all bucket members minus the probe party itself.

III. SELF-CALIBRATING FELLEGI-SUNTER PARAMETER ESTIMATION

III.A. The Fellegi-Sunter Model

The Fellegi-Sunter (1969) probabilistic record linkage framework assigns a match weight to each comparison vector $\gamma = (\gamma_1, \dots, \gamma_k)$ where $\gamma_i \in \{0, 1\}$ indicates agreement on attribute i . The log-likelihood ratio weight for attribute i is:

$$w_i = \gamma_i \times \ln(m_i / u_i) + (1 - \gamma_i) \times \ln((1 - m_i) / (1 - u_i))$$

where $m_i = P(\gamma_i = 1 \mid \text{true match})$ and $u_i = P(\gamma_i = 1 \mid \text{true non-match})$.

In all prior art systems, m and u are hardcoded. The present invention learns them automatically.

III.B. Expectation-Maximization Algorithm

The ``EMAlgorithmService`` implements unsupervised EM parameter estimation operating on ten attributes across four categories:

Name attributes (indices 0–1): `firstName` (IDX_FIRST_NAME=0), `lastName` (IDX_LAST_NAME=1)

Temporal (index 2): dateOfBirth (IDX_DOB=2)

Identifiers (index 3): taxId/EIN/SSN (IDX_TAX_ID=3)

Contact (indices 4–5): email (IDX_EMAIL=4), phone (IDX_PHONE=5)

Organization (index 6): organizationName (IDX_ORG_NAME=6)

Geographic (index 7): postalCode (IDX_POSTAL=7)

Phonetic (indices 8–9): phonetic firstName (IDX_PHONETIC_FN=8), phonetic lastName (IDX_PHONETIC_LN=9)

Algorithm Steps:

1. Sampling: N random pairs are drawn from Neo4j golden party records using reservoir sampling. Default N=5,000 pairs.
2. Feature vector computation: For each pair (A, B), a 10-dimensional binary agreement vector γ is computed. Agreement is defined per attribute: exact match for identifiers, Jaro-Winkler ≥ 0.85 for names, phonetic match for phonetic attributes.
3. Initialization: m and u vectors are initialized to domain-expert prior values:

```
Default m (P(agree | match)): [0.95, 0.92, 0.88, 0.99, 0.85, 0.82, 0.90, 0.75, 0.89, 0.86]
Default u (P(agree | non-match)): [0.15, 0.10, 0.02, 0.01, 0.03, 0.04, 0.08, 0.12, 0.14, 0.09]
```

4. E-Step: For each pair i, compute the posterior probability that it is a true match:

```
log_match_i = ln( $\pi$ ) +  $\sum_k [\gamma_{ik} \times \ln(m_k) + (1-\gamma_{ik}) \times \ln(1-m_k)]$ 
log_nomatch_i = ln(1- $\pi$ ) +  $\sum_k [\gamma_{ik} \times \ln(u_k) + (1-\gamma_{ik}) \times \ln(1-u_k)]$ 
maxLog = max(log_match_i, log_nomatch_i)
P(match_i) = exp(log_match_i - maxLog) / [exp(log_match_i - maxLog) + exp(log_nomatch_i - maxLog)]
```

The 'maxLog' offset ensures numerical stability in log-space computation.

5. M-Step: Update m, u, and π from posterior-weighted observations:

```
m_k_new =  $\sum_i [P(\text{match}_i) \times \gamma_{ik}] / \sum_i [P(\text{match}_i)]$ 
u_k_new =  $\sum_i [(1 - P(\text{match}_i)) \times \gamma_{ik}] / \sum_i [(1 - P(\text{match}_i))]$ 
 $\pi$ _new = mean(P(match_i))
```

6. Convergence: Repeat E and M steps until ' $\|m_{\text{new}} - m_{\text{old}}\|_1 + \|u_{\text{new}} - u_{\text{old}}\|_1 < \epsilon$ ' ($\epsilon = 0.001$) or maximum iterations (default 100) is reached.
7. Parameter Storage: Results are stored as 'MUParameters(double[] m, double[] u, double pi, String partyType, LocalDateTime learnedAt)' in a per-party-type concurrent hash map.
8. Scheduled Re-estimation: The '@Scheduled(cron = "0 0 2 * * *")' annotation triggers nightly re-estimation at 02:00 UTC for all party types with sufficient data.

III.C. Sanity Guard

Before accepting EM-estimated parameters in the scoring pipeline, the system applies the guard:

```
if (m > u && m > 0.5 && u < 0.5) return new double[]{m, u};  
return prior; // revert to prior if EM values unsensible
```

This prevents statistically aberrant EM results (e.g., from sparse data or early iterations) from degrading match quality.

IV. THREE-STAGE CASCADING MATCH PIPELINE

IV.A. Architecture

`MatchingEngine.scoreAgainstPool()` applies three stages in sequence:

Stage 1 — Deterministic Matching:

Critical unique identifiers are compared first: SSN, Tax ID, EIN, DUNS number, LEI, passport number, and national ID. If any identifier matches exactly between the probe and a candidate, the score is set to 1.0 (definite match) and deterministic method is recorded. An additional deterministic rule fires when the probe and candidate share both `sourceSystem` and `sourceSystemId`, preventing duplicate ingest from the same source.

Stage 2 — Probabilistic Fellegi-Sunter Scoring:

`ProbabilisticMatcher.scoreIndividual()` or `scoreOrganization()` is invoked for non-deterministic candidates. The scorer applies 20+ similarity algorithms across name, date-of-birth, contact, and identifier attributes, maps each comparison outcome to a Fellegi-Sunter log-likelihood contribution, and normalizes the total to [0, 1].

The 20+ similarity algorithms include:

- Jaro-Winkler similarity for first name, last name, and organization name
- Token Sort Ratio (alphabetically sorted tokens before JW) for word-order invariance
- Token Set Ratio (Jaccard over token sets + JW) for abbreviation handling
- Bigram Jaccard similarity (2-gram character overlap)
- Trigram Jaccard similarity (3-gram character overlap)
- Composite name similarity (max of multiple metrics with phonetic boost)
- Damerau-Levenshtein / Optimal String Alignment distance (four operations: substitution, insertion, deletion, transposition)
- TF-IDF cosine similarity over character n-gram vectors
- Double Metaphone phonetic encoding
- Nickname equivalence lookup (90+ name variant groups)

Stage 3 — AI-Enhanced Disambiguation (Novel):

If the probabilistic score falls in the uncertain range [0.5, 0.9] AND the active matching rule has `useAIEnhancement = true`, the system calls `AIEnhancedMatcher.score(partyA, partyB)`. The matcher constructs a structured prompt containing both records' key attributes and requests:

`SCORE:<0.00-1.00>|REASON:<brief explanation>`

The final score is a weighted blend: $0.6 \times \text{probabilistic_score} + 0.4 \times \text{ai_score}$. The method is recorded as `AI\_ENHANCED`.

IV.B. Threshold-Driven Action Routing

The top-scoring candidate and its score determine the ingest action:

Score Range	Action	Meaning
-------------	--------	---------

---	---	---
-----	-----	-----

≥ 0.95	`AUTO_LINK`	Definite match — link to candidate's golden cluster
-------------	-------------	---

0.75 – 0.94	`SEND_TO_STEWARD`	Probable match — human review required
-------------	-------------------	--

< 0.75	`CREATE_NEW`	No match — new independent entity
----------	--------------	-----------------------------------

Thresholds are configurable per `MatchingRule` entity, allowing different sensitivity for different entity types or source systems.

V. PROVISIONAL GOLDEN IDENTITY MANAGEMENT (NOVEL INVENTION)

V.A. The Null Golden State Problem

In all prior art MDM systems, a party record pending steward review exists in a null golden state — it has been ingested but has no golden identifier. During review (which may take days to weeks), the record is invisible to:

- Downstream applications querying by golden ID
- The survivorship engine (cannot compute a golden view without a golden ID)
- The timeline service (events indexed by golden ID cannot be recorded)
- The blocking index (cannot be found as a candidate for future matches)

This null state violates the core MDM guarantee of universal entity availability.

V.B. The Provisional Golden Identity

The present invention eliminates the null golden state entirely through the **Provisional Golden Identity** pattern:

For SEND_TO_STEWARD matches (score 0.75–0.94):

9. A provisional golden ID is generated: `newGoldenId = generateGoldenId()`.

10. The incoming party is assigned this provisional ID:

`incoming.setGoldenRecordId(provisionalGoldenId)`.

11. The party is saved to Neo4j with `isGolden = false` (source record designation).
12. `GoldenRecordService.createNewGoldenRecord(provisionalGoldenId, List.of(saved))` is immediately called, creating a complete golden record with survivorship rules applied.
13. The provisional golden party is added to the blocking index: `indexParty(saved)`.
14. A steward task is created with `candidateIds = [existingCandidateGoldenId, provisionalGoldenId]`.

During the review period:

The provisional record is fully available: it has a golden ID, a golden record, appears in blocking index candidates, and receives timeline events.

Upon steward APPROVE_MERGE:

`StewardService.executeResolution()` calls `mergeGoldenRecords(survivingGoldenId, provisionalGoldenId)`. The party is re-pointed to the surviving golden ID. The provisional golden ID is retired.

Upon steward REJECT_MERGE:

The provisional golden ID becomes the permanent golden ID. The record remains its own independent entity. No additional action required.

This design guarantees: **every party record has a golden identifier at all times, from the moment of first database commit.**

V.C. Client-Supplied Golden Identity (Migration Path)

When migrating existing data from legacy systems, clients may supply their own golden identifiers:

```
POST /api/v1/parties/ingest
{
  "goldenRecordId": "ORACLE-SEQ-10004231",
  ...
}
```

When `goldenRecordId` is non-blank in the ingest payload, the matching engine is bypassed entirely and the supplied identifier is used as-is. This enables zero-downtime migration of existing identity systems into Averio MDM.

VI. POST-UPDATE PROBABILISTIC CLUSTER DRIFT DETECTION (NOVEL INVENTION)

VI.A. The Drift Problem

Entity attributes are not static. A phone number may be corrected, an address updated, a tax ID completed, or a name formally changed. After any such update, the statistical basis on which a party was originally clustered may no longer hold. No known MDM system detects or responds to this condition automatically.

VI.B. Drift Detection Algorithm

`PartyService.reEvaluatePartyPlacement(Party party, String updatedBy)` is invoked after every `updateParty()` operation. The algorithm:

Step 1 — Intra-Cluster Scoring:

Retrieve all sibling parties sharing the same `goldenRecordId`, excluding the updated party itself.

```
List<Party> siblings = partyRepository.findByGoldenRecordId(currentGoldenId)
    .stream().filter(s -> !s.getGlobalId().equals(party.getGlobalId()))
    .collect(Collectors.toList());
```

Score the updated party against each sibling using `matchingEngine.findMatches(party, siblings, null)`. Extract `bestSiblingScore`.

Step 2 — Drift Threshold Test:

If `bestSiblingScore >= REVIEW\_THRESHOLD (0.75)`, no drift is detected. The party still statistically belongs in its cluster. Return early. **No action taken for the common case.**

Step 3 — External Candidate Search (Drift Confirmed):

If drift is confirmed (`bestSiblingScore < 0.75`), execute the full blocking + scoring pipeline against the global golden record pool:

```
MatchingEngine.MatchResult externalResult =
    matchingEngine.findMatchesWithBlocking(party, null);
```

Filter results to candidates belonging to a *different* golden cluster than the current one.

Step 4 — Drift Remediation (Three-Branch):

| Condition | Action | Timeline Events |

|---|---|---|

| `externalScore >= 0.95` | `reassignPartyToGolden()` — auto-reassign |
| `PARTY\_LEFT\_CLUSTER` on old, `PARTY\_JOINED\_CLUSTER` on new |

| `externalScore 0.75–0.94` | `createMatchReviewTask()` — send to steward |
| `CLUSTER\_DRIFT\_REVIEW\_REQUESTED` |

| No external match | `detachToNewGolden()` — create new golden |
| `PARTY\_DETACHED\_FROM\_CLUSTER`, `GOLDEN\_CREATED\_AFTER\_DRIFT` |

Step 5 — Refresh Old Golden:

Regardless of branch taken, ``goldenRecordService.refreshGoldenRecord(currentGoldenId)`` is called to update the surviving cluster's golden record without the departed party.

VI.C. Timeline Event Architecture for Drift

All cluster membership changes are recorded as immutable ``TimelineEvent`` documents in Cosmos DB, partitioned by ``/entityId`` (golden record ID):

- ``PARTY_LEFT_CLUSTER``: Recorded on the **old** golden ID. Contains old score, new golden ID, triggered by.
- ``PARTY_JOINED_CLUSTER``: Recorded on the **new** golden ID. Contains match score, match method.
- ``CLUSTER_DRIFT_REVIEW_REQUESTED``: Recorded on the **old** golden ID. Contains provisional new golden ID, steward task ID.
- ``PARTY_DETACHED_FROM_CLUSTER``: Recorded on the **old** golden ID.
- ``GOLDEN_CREATED_AFTER_DRIFT``: Recorded on the **new** golden ID.

This creates a complete, auditable history of cluster membership changes that is queryable by golden ID, enabling compliance reporting and point-in-time reconstruction of cluster state.

VII. TRANSITIVE CLUSTER MERGING VIA UNION-FIND

``ClusterMergeService`` resolves transitive closure in matching chains. If record A matches record B ($\text{score} \geq \text{threshold}$) and record B matches record C, then A, B, and C must belong to the same cluster even if A and C have never been directly compared.

The Union-Find (Disjoint Set Union) algorithm is applied:

15. Initialize each record as its own cluster representative.
16. For each pair (A, B) above the link threshold, call ``union(A, B)``.
17. ``union(a, b)`` uses path compression and union-by-rank for $O(\alpha(N))$ time per operation.
18. ``getClusters()`` returns a map from representative ID to all member IDs.

This ensures: **no two records that are transitively matched remain in separate golden clusters.**

VIII. QUERY-TIME SURVIVORSHIP RULE ENGINE

VIII.A. Architecture

Golden records in Averio MDM are not pre-computed and stored as static entities. Instead, they are assembled from source records at query time by applying survivorship rules. This enables:

- Rule changes to take effect immediately without re-computing stored records

- Per-view golden record construction for different business units or regulatory requirements
- Full auditability of which source record contributed each attribute

VIII.B. Rule Types

The survivorship engine (`SurvivorshipEngine.buildGoldenRecord()`) applies six rule types per attribute:

19. **SOURCE_PRIORITY**: An ordered list of source systems defines preference. The attribute value from the highest-priority available source wins.
20. **MOST_RECENT**: The attribute value with the most recent `sourceLastUpdated` timestamp wins.
21. **MOST_FREQUENT**: The attribute value that appears most frequently across source records wins (majority vote).
22. **SUPREMACY**: A designated "supremacy source" system always wins regardless of recency or frequency.
23. **NON_NULL**: The first non-null value across sources, ordered by source priority, wins.
24. **LONGEST**: The longest string value wins (useful for address fields where longer usually means more complete).

VIII.C. View-Scoped Golden Records

When a `viewId` parameter is supplied, `GoldenRecordService.getGoldenRecordForView()` loads `SurvivorshipRule` entities filtered to that view. Different views may apply different rule sets to the same underlying source data, yielding different golden record values for different business consumers — all from a single authoritative data store.

IX. HUMAN-IN-THE-LOOP ML RETRAINING PIPELINE

IX.A. Feature Capture at Decision Time

When a steward makes a merge/reject decision on a steward task, `MLMatchingService.captureDecision()` is invoked. The system immediately extracts eleven core features from the current state of both party records:

```
nameSimilarity, dobExactMatch, taxIdExactMatch, emailMatch, phoneMatch,
addressSimilarity, dunsMatch, leiMatch, nationalIdMatch,
sourceSystemDiversity, partyTypeMatch
```

These features are stored with the steward's label (`MATCH` / `NO_MATCH`) as an immutable `MatchingFeedback` document in Cosmos DB.

Critical innovation: Features are captured at decision time rather than queried at training time. This is necessary because party attributes evolve; the feature vector that informed the steward's decision is the ground truth, not the feature vector that would be computed from current attribute values months later. This enables **replay-free retraining** — the training set can be rebuilt from stored feedback documents without requiring original party record state.

IX.B. Automatic Retraining Trigger

After each feedback capture, the system checks:

```
if (feedbackCount >= MIN_EXAMPLES && feedbackCount % RETRAIN_EVERY == 0) {  
    retrainModel(entityType);  
}
```

Default values: `MIN\_EXAMPLES = 5`, `RETRAIN\_EVERY = 5`. This means the model retrains every five new labeled examples once a minimum threshold is met.

IX.C. Logistic Regression Training

The `MLMatchingService.trainModel()` method implements logistic regression:

- Feature matrix: $X \in \mathbb{R}^{(n \times d)}$, n = example count, d = 11 features
- Target vector: $y \in \{0, 1\}^n$
- Weight vector: $w \in \mathbb{R}^{(d+1)}$ (d features + bias)
- Gradient descent: up to 500 iterations
- Learning rate: $\eta = 0.1$
- L2 regularization: $\lambda = 0.01$ (applied to feature weights, not bias)
- Prediction: $\hat{y} = \sigma(w^T x)$ where σ is the sigmoid function

After training, the system computes accuracy, precision, recall, F1 score, and per-feature importance ($|w_i| / \sum |w_j|$) with direction classification (POSITIVE / NEGATIVE / NEUTRAL).

X. NAME NORMALIZATION SUBSYSTEM

X.A. Organization Name Normalization

`NameNormalizerService.normaliseOrg()` applies:

25. Legal suffix stripping: 30+ legal entity suffixes in multiple languages are stripped using longest-match-first to prevent partial matches ("Corp" matching before "Corporation").
26. Punctuation normalization: Commas, dots, and other punctuation are replaced with spaces.
27. Whitespace collapsing: Multiple consecutive spaces are reduced to single spaces.
28. Lowercase conversion.

Example: `"IBM Corporation, Inc."` → `"ibm"`, ensuring that `"IBM"`, `"IBM Corp"`, `"IBM Corporation"`, and `"IBM Inc."` all produce identical blocking keys.

X.B. Nickname Equivalence Service

`NicknameService` maintains 90+ equivalence groups covering common English given names with international variants. Methods:

- `variants(String firstName)`: Returns all equivalent forms including the input
- `areEquivalent(String a, String b)`: Boolean equivalence test
- `similarity(String a, String b)`: Returns 1.0 for equivalent names, 0.0 otherwise

The impact: `"Bob Smith"` and `"Robert Smith"` generate overlapping blocking keys (Strategy 1b) and score higher in the probabilistic matcher via the nickname similarity feature.

XI. IN-APPLICATION TEST LABORATORY (NOVEL OPERATIONAL FEATURE)

The Averio MDM platform includes a built-in automated testing framework (`com.averio.mdm.testing`) that executes inside the live application against real infrastructure, enabling continuous verification without a separate testing environment.

Seven test suites:

- 29. API_HEALTH: Eight connectivity checks across all data stores and services
- 30. MATCHING: Seven in-memory matching algorithm tests (zero database I/O)
- 31. BLOCKING: Five blocking key and index operation tests (zero database I/O)
- 32. SURVIVORSHIP: Three survivorship rule application tests
- 33. GOLDEN_RECORD: Four golden record service tests with Cosmos DB persistence
- 34. TIMELINE: Four timeline event persistence and ordering tests with per-test cleanup
- 35. REGRESSION: Five end-to-end ingest pipeline scenarios with Neo4j + Cosmos DB persistence and automatic test data cleanup

All suite dependencies use `@Autowired(required=false)` with null-guard returning `SKIPPED` status for unavailable services, enabling partial test runs in degraded environments.

Test data is isolated via `sourceSystem = "TEST_LAB"` and `sourceSystemId = "TEST-{runId}-{seq}"` markers, with cleanup IDs tracked per result for targeted deletion.

CLAIMS

INDEPENDENT CLAIMS

Claim 1. A computer-implemented method for entity resolution in an enterprise master data management system comprising:

- receiving a party record from a source system, the party record comprising at least a name field and at least one identifier field;
- generating a plurality of blocking keys from the party record using nine independent blocking key strategies applied in union, the strategies comprising: (a) phonetic encoding of individual name tokens; (b) nickname variant phonetic encoding; (c) full collapsed-name phonetic encoding; (d) first initial plus last-token phonetic encoding; (e) date-of-birth year-month plus name initial; (f) tax identifier prefix; (g) phone number suffix; (h) email domain plus name initial; and (i) exact high-cardinality organizational identifiers;
- retrieving, from an inverted index, a candidate pool comprising all party records sharing at least one blocking key with the received party record;
- scoring each candidate in the candidate pool using a probabilistic Fellegi-Sunter model parameterized by m-probabilities and u-probabilities that are automatically estimated by an Expectation-Maximization algorithm running on the same system without requiring human-labeled training pairs;
- routing the top-scoring candidate to one of: auto-link, steward review, or new entity creation based on configurable threshold values; and
- if routing to steward review, assigning a provisional globally unique golden identifier to the received party record before committing any database transaction, thereby eliminating a null golden identity state.

Claim 2. A computer-implemented method for post-update cluster drift detection in an enterprise master data management system comprising:

- detecting an attribute update operation on a party record that is a member of a golden cluster comprising a plurality of party records sharing a common golden record identifier;
- in response to the detected update, computing match scores between the updated party record and all sibling party records in the golden cluster;
- if the highest intra-cluster match score falls below a first threshold value, determining that statistical drift has occurred;
- upon drift determination, executing a full blocking-key-indexed candidate search across all golden record clusters in the system to identify external candidate matches for the updated party record;
- if an external candidate match score meets or exceeds a second threshold value, automatically reassigning the party record to the external candidate's golden cluster and recording first and second timeline events on the respective golden record identifiers; and
- if no external match meets any threshold, detaching the party record from its current golden cluster and creating a new golden record identifier for the detached party, recording a detachment timeline event and a new golden creation timeline event.

Claim 3. A system for self-calibrating probabilistic record linkage parameter estimation in a master data management platform comprising:

- at least one processor executing an Expectation-Maximization algorithm that estimates, without labeled training data, m-probabilities representing the probability of attribute agreement given a true match and u-probabilities representing the probability of attribute agreement given a true non-match, for each of a plurality of entity attributes;
- a parameter store maintaining estimated m-probability and u-probability vectors per party type with a timestamp indicating estimation time;
- a sanity guard module that validates estimated parameters against domain constraints, reverting to prior values when estimated parameters fail validation;
- a scheduler configured to trigger automatic re-estimation on a recurring schedule; and
- a probabilistic scoring engine that incorporates the estimated parameters into Fellegi-Sunter log-likelihood ratio computations.

Claim 4. A computer-implemented method for human-in-the-loop machine learning model maintenance in an entity resolution system comprising:

- at a time of a human expert decision regarding a candidate record pair, capturing a feature vector representing current attribute comparison values between the two records;
- persisting the captured feature vector with the human expert's match or no-match label as an immutable feedback record;
- upon accumulation of a threshold count of feedback records, automatically retraining a logistic regression model using the stored feature vectors and labels;
- computing trained model performance metrics including accuracy, precision, recall, and F1 score; and
- replacing the active scoring model with the retrained model if performance metrics meet minimum acceptance criteria.

Claim 5. A computer-implemented method for provisional identity assignment in a master data management system comprising:

- receiving an entity record requiring human expert review before permanent golden cluster assignment;
- prior to any human expert review, generating a provisional globally unique identifier for the entity record;
- creating a complete golden record associated with the provisional identifier, comprising survivorship-rule-applied attribute values derived from the single entity record;
- indexing the entity record in a blocking index using the provisional golden identifier;
- making the entity record available to downstream applications via the provisional golden identifier during the entire review period;

- upon human expert approval of a merge with a candidate cluster, migrating the entity record to the candidate cluster's golden identifier and retiring the provisional identifier; and
- upon human expert rejection of all proposed merges, promoting the provisional golden identifier to a permanent golden identifier without any additional operations.

DEPENDENT CLAIMS

Claim 6. The method of Claim 1, wherein the probabilistic scoring further comprises a third AI-enhancement stage that is conditionally invoked when the probabilistic score falls within a predetermined ambiguous range, the AI-enhancement stage comprising: constructing a structured prompt containing attribute data from both party records; submitting the prompt to a large language model; parsing a numeric score from the model response; and computing a final blended score as a weighted combination of the probabilistic score and the AI score.

Claim 7. The method of Claim 1, wherein generating blocking keys using nickname variant phonetic encoding comprises: looking up a given name in a pre-stored table of equivalence groups, each group comprising a canonical name and all culturally and historically equivalent variant names; and generating a phonetic blocking key for each variant name in the matching equivalence group.

Claim 8. The method of Claim 1, wherein the inverted index is maintained in a first concurrent hash map mapping blocking key strings to sets of global identifiers, and a forward index is maintained in a second concurrent hash map mapping global identifiers to sets of blocking key strings, enabling $O(1)$ removal of all blocking index entries for a party record upon deletion or merge.

Claim 9. The method of Claim 2, wherein the timeline events comprise structured documents persisted to a distributed document database partitioned by golden record identifier, each document comprising: an event type identifier; a previous state representation; a new state representation; a timestamp; an actor identifier; and a map of event-specific metadata.

Claim 10. The method of Claim 3, wherein the Expectation-Maximization algorithm comprises: drawing a random sample of party record pairs from a graph database; computing a binary agreement vector for each pair across ten attributes; in an E-step, computing posterior match probabilities using numerically stable log-space computation with a maximum-log offset to prevent floating-point underflow; and in an M-step, updating m and u estimates as posterior-weighted averages of agreement indicators.

Claim 11. The system of Claim 3, further comprising: a query-time survivorship rule engine that assembles a golden record from underlying source records at retrieval time by applying one of: source priority ordering; most-recent timestamp selection; majority vote across sources; supremacy source designation; first non-null value selection; or longest

string selection per attribute; wherein the assembled golden record is not pre-computed but is generated fresh on each retrieval, enabling rule configuration changes to take effect immediately.

Claim 12. The method of Claim 4, wherein the feature vector captures attribute comparison values at the time of human expert decision rather than at training time, preserving the statistical basis for the decision independent of subsequent record mutations.

Claim 13. The method of Claim 5, wherein the provisional golden identifier is assigned using a random number generator producing a zero-padded fixed-length numeric string, and wherein a client-supplied identifier in any format may optionally override the auto-generated identifier during migration from legacy systems.

Claim 14. A computer-implemented method for transitive entity cluster formation comprising: computing pairwise match scores between entity records to produce a set of pairs above a link threshold; applying a Union-Find algorithm with path compression and union-by-rank to identify transitive closure clusters; wherein any two records that are transitively connected through any chain of pairwise matches are assigned to the same golden cluster regardless of whether they have been directly compared.

Claim 15. The method of Claim 1, wherein the probabilistic scoring applies twenty or more similarity algorithms to entity attributes, the algorithms comprising at least: Jaro-Winkler similarity; token sort ratio; token set ratio; bigram Jaccard similarity; trigram Jaccard similarity; Damerau-Levenshtein optimal string alignment distance; Double Metaphone phonetic encoding; TF-IDF cosine similarity over character n-gram vectors; and nickname equivalence lookup; wherein multiple algorithm outputs are combined per attribute using Fellegi-Sunter log-likelihood weighting.

ABSTRACT

An enterprise Master Data Management (MDM) platform implements adaptive entity resolution through five novel technical innovations. First, a nine-strategy union blocking engine generates blocking keys using phonetic encoding, nickname expansion, date-of-birth hashing, identifier prefixes, geographic encoding, and exact identifier matching applied in union, reducing pairwise comparison complexity from $O(N^2)$ to $O(N \times k)$. Second, a self-calibrating Expectation-Maximization algorithm autonomously estimates Fellegi-Sunter m and u probability parameters from unlabeled data on a recurring schedule, eliminating manual calibration. Third, a three-stage cascading match pipeline combines deterministic, probabilistic, and AI-enhanced (Large Language Model) matching with threshold-driven routing to auto-link, steward review, or new entity creation. Fourth, a provisional golden identity management system assigns a globally unique identifier to every party record before any database commit — including records

pending steward review — eliminating the null golden state condition present in all prior-art MDM systems and guaranteeing continuous downstream data availability. Fifth, a post-update cluster drift detector re-evaluates party cluster membership after every attribute update, automatically reassigning, escalating to steward review, or detaching to a new cluster when statistical drift is confirmed. All cluster membership changes are recorded as immutable timeline events enabling full audit trail reconstruction and compliance reporting.

SEQUENCE LISTING / COMPUTER PROGRAM LISTINGS

Computer program listings for the key algorithmic innovations are appended as Exhibits A through G:

- Exhibit A: `BlockingKeyService.java` — Nine-strategy blocking key generation
 - Exhibit B: `EMAlgorithmService.java` — Expectation-Maximization parameter estimation
 - Exhibit C: `MatchingEngine.java` — Three-stage cascading match pipeline
 - Exhibit D: `ProbabilisticMatcher.java` — Fellegi-Sunter probabilistic scorer
 - Exhibit E: `PartyService.java` — Ingest pipeline and drift detection
 - Exhibit F: `GoldenRecordService.java` — Query-time survivorship assembly
 - Exhibit G: `MLMatchingService.java` — Human-in-the-loop logistic regression
-

INVENTOR DECLARATION

I hereby declare that:

(a) Each undersigned inventor believes the named inventors are the original and first inventors of the subject matter which is claimed and for which a patent is sought on the invention titled:

"ADAPTIVE MULTI-STRATEGY ENTITY RESOLUTION SYSTEM WITH PROBABILISTIC CLUSTER DRIFT DETECTION, PROVISIONAL GOLDEN IDENTITY MANAGEMENT, AND SELF-CALIBRATING FELLEGI-SUNTER PARAMETER ESTIMATION FOR ENTERPRISE MASTER DATA MANAGEMENT"

(b) The undersigned inventors acknowledge the duty to disclose information that is material to patentability as defined in 37 C.F.R. § 1.56.

Inventor 1:

Name: Suvojeet Pal

Residence: [Address]

Citizenship: [Citizenship]

Signature: _____ Date: _____

Inventor 2:

Name: Rakhi Chatterjee

Residence: [Address]

Citizenship: [Citizenship]

Signature: _____ Date: _____

INFORMATION DISCLOSURE STATEMENT (IDS) — PRIOR ART

The following prior art is disclosed for examiner consideration:

U.S. Patents

- US 7,853,573 B2 — "System and method for data integration" (IBM InfoSphere)
- US 8,285,719 B2 — "Entity resolution in master data management"
- US 9,330,169 B2 — "Probabilistic record linkage with feedback learning"
- US 10,437,882 B2 — "Dynamic survivorship rules for master data"

Non-Patent Literature

36. Fellegi, I.P. and Sunter, A.B. (1969). "A theory for record linkage." *Journal of the American Statistical Association*, 64(328), 1183-1210.
37. Winkler, W.E. (1988). "Using the EM algorithm for weight computation in the Fellegi-Sunter model of record linkage." *Proceedings of the Section on Survey Research Methods*, ASA, 667-671.
38. Larsen, M.D. and Rubin, D.B. (2001). "Iterative automated record linkage using mixture models." *Journal of the American Statistical Association*, 96(453), 32-41.
39. Damerau, F.J. (1964). "A technique for computer detection and correction of spelling errors." *Communications of the ACM*, 7(3), 171-176.
40. Christen, P. (2012). *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer.
41. Köpcke, H. and Rahm, E. (2010). "Frameworks for entity matching: A comparison." *Data & Knowledge Engineering*, 69(2), 197-210.

Distinguishing Features Over Prior Art

The present invention is distinguished from all cited prior art in at least the following respects:

42. Over static probabilistic systems (US 9,330,169): The present invention's EM algorithm automatically re-estimates Fellegi-Sunter parameters on a schedule without human-labeled training pairs. The cited reference requires labeled training sets and does not provide automatic scheduled re-estimation.
43. Over blocking-based systems: No prior art reference discloses nine independent blocking strategies applied in union with a dual (inverted + forward) concurrent hash map structure. The cited references apply two to four strategies at most.
44. Over survivorship rule systems (US 10,437,882): The cited reference pre-computes and stores golden record attribute values. The present invention assembles golden records at query time from source records, enabling instant rule changes without data re-computation and supporting multiple view-scoped golden records from the same underlying data.
45. Over all prior MDM systems: No prior art discloses the Provisional Golden Identity pattern that eliminates the null golden state. All known commercial MDM systems (IBM InfoSphere MDM, Informatica MDM, SAP Master Data Governance, TIBCO EBX, Reltio Cloud) allow party records pending steward review to exist without a golden identifier.
46. Over all prior MDM systems: No prior art discloses post-update probabilistic cluster drift detection. The concept of automatically re-evaluating cluster membership when party attributes change is wholly novel to the field of entity resolution.
47. Over AI-based matching systems: No prior art discloses conditional three-stage cascading that falls back to LLM-based disambiguation only for the statistically ambiguous score range [0.5, 0.9], preserving deterministic and probabilistic accuracy while using AI only as a targeted tiebreaker.

End of Patent Application

> **ATTORNEY NOTE:** This document has been prepared for filing under 35 U.S.C. § 111(a) as a non-provisional utility patent application. Filing fees apply per 37 C.F.R. § 1.16. An Information Disclosure Statement (IDS) should be filed concurrently with this application per 37 C.F.R. § 1.97. The inventors should execute the Inventor Declaration (AIA Form PTO/AIA/01) and an Assignment document (PTO/AIA/75) prior to filing. Drawings referenced in the Brief Description of Drawings section should be prepared as black-ink line drawings conforming to 37 C.F.R. § 1.84 and filed as separate exhibits. Consider filing a concurrent Patent Cooperation Treaty (PCT) application under the Patent Cooperation Treaty for international protection in jurisdictions where Averio MDM will be commercialized.